

Scalable Modeling of Software Product Line Variability

Baris Can Kasikci¹, Semih Bilgen²

¹ Siemens Corporate Technology
Siemens A.S. Istanbul, 34870 Istanbul, Turkey
baris.kasikci@siemens.com

² Middle East Technical University, Department of Electrical and Electronics Engineering
06531 Ankara, Turkey
semih-bilgen@metu.edu.tr

Abstract

Modeling and tracking of variability is essential for the success of a Software Product Line (SPL) but raises an apparent scalability problem. The reason for this is that variability is generally spread across multiple artifacts constituting the product line (PL) and therefore a mechanism to manage it can easily grow out of control if not structured properly. This work introduces concerns, a concept for modeling variability in an SPL. The process of modeling variability using concerns is presented and its relation to PL scalability is discussed. Furthermore, novel evaluation criteria are introduced to determine the usefulness of concerns in modeling PL variability.

1. Introduction

The primary aim of SPL's is to increase the amount of proactive reuse and therefore allow a sound return on investment in their creation. PL's focus on producing individualized products of a family at a lower cost. In order to achieve this, a fundamental amount of commonality is essential. PL context is driven by the commonality information and opportunities for financial success.

Variability in a PL is generally spread throughout several artifacts such as features, requirements, software design, source code and test cases. Traditional PL approaches tend to trace variability by merely using features and relating feature variation information to other artifacts. However other artifacts generally possess an intrinsic nature in terms of variability and although this is related with the variability of features, its separate treatment is advocated [1], [2]. Variability

information is what enables the creation of new systems out of the PL. Novel system specification using variability spread across numerous PL assets allows greater opportunities of configurability and fine-tuning.

Several approaches to variability modeling such as the Orthogonal Variability Model (OVM) [1] and decisions [2], [3] treat variability at a different level and construct a separate variability model. However separate and thorough treatment of variability comes with its cost. Evidently such an approach is demanding in terms of the overhead brought to the PL process, since separate techniques and traceability mechanisms are necessary. A scalability problem is likely to arise with the increasing modeling technique overhead.

In this work, after a review of relevant literature, concerns are introduced as a scalable means for modeling PL variability. Two assessment criteria are introduced and the proposed approach is evaluated in terms of its advantages in modeling variability compared to a conventional method.

2. Product lines and features

Many approaches aim constructing a PL. Few of them that relate to this work and explicitly mention variability modeling strategies will be briefly mentioned. Software product line engineering (SPLE) grew from domain engineering (DE) methodologies like FODA [4]. FODA introduced the notion of features for modeling commonality and variability in PL's and bases the entire domain engineering efforts and artifact parameterization on features.

However, FODA did not concentrate on component construction and application engineering. Feature Oriented Reuse Method (FORM) [5], the successor of FODA, focuses on application engineering in addition

to domain engineering and the construction of reusable components. FORM introduces principles to build these components and parameterizes its artifacts and components using features just like FODA.

A more recent PL approach is Product Line UML based Software Engineering (PLUS) [6]. PLUS uses UML constructs as use cases, class diagrams and collaboration diagrams to model artifacts such as domain requirements, relations and the communication between components.

A feature is defined as a prominent part or characteristic of an entity [7]. In all the mentioned approaches, features play a critical role. Feature modeling evolved over the time and several novel techniques and notations were introduced. A collection and comparison of a broad range of feature modeling approaches is available in [8]. For the purposes of variability modeling techniques used in this work, it is helpful to consider the feature model in Figure 1. Below, items of the model are referred to as feature groups (upper case items) and features themselves (lowercase items), however, a general view as variation points and variants respectively is more applicable for the purposes of this work (with the exception of part a) of Figure 1 which denotes commonality).

Modeling technique in Figure 1 is basically that of FORM, with an additional “or” group as represented in [9]. In Figure 1, part a) denotes commonality of features “a” and “b” (mandatory features).

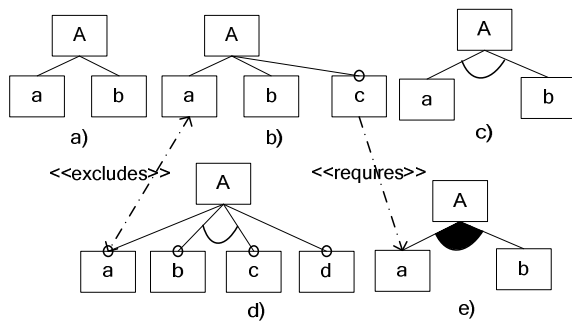


Figure 1 - Feature model and feature relationships

In part b), two common features “a” and “b” and an optional “c” are represented. The relations in part c) depict that “a” and “b” are alternatives of each other. In part d), four optional features “a”, “b”, “c” and “d” can be observed where “b” and “c” are alternatives of each other. The last item, namely part e) denotes logical “or” operation for the features “a” and “b”. Note *requires/excludes* relationships and their directions. *Excludes* tend to be bidirectional whereas *requires* can be both unidirectional and bidirectional.

3. Scalability in product line modeling

Scalability of a modeling approach could be defined as its ability to handle the growing or shrinking amount of assets and elements that make up the model, both with respect to size and time requirements. Scalability of a modeling technique has to satisfy certain requirements.

The first requirement pointed out in [2] states that in the particular context of variability, a scalable approach is one that is able to manage variability when it is large or small (upwards/downwards scalability). This means a scalable variability modeling mechanism is one that is applicable to a PL of any size.

The second requirement of a scalable approach is that the introduction of new modeling elements should not have a negative effect on the existing model’s coherence. It has been stated in [10] that with the progressive introduction of new systems into the PL, variability handling mechanisms of these PL’s were not capable of merging the new information into the existing model. It has also been stated that this gradual insertion led to the deterioration of the existing variability modeling infrastructure.

The final requirement that is addressed in this work regarding the scalability of a modeling approach is expressed in [11]. According to this, scalability should not only be understood as the ability to handle small or large scale problems but that it is also related to the selective and incremental application of a particular modeling technique. The modeling approach presented in this work aims to fulfill all three requirements of scalability mentioned in this section.

4. Variability modeling and tracking

FODA places features as the fundamental means of domain analysis. FORM follows a similar approach and takes one step further by incorporating reusable component parameterization via features. PLUS, on the other hand establishes links of traceability of features to use cases and software objects realizing the related features.

The common approach taken by most of the SPLE methods is their usage of features in modeling variability. However as indicated in [1], the management of variability in an SPL is dependent on the inputs from product managers, software architects, developers and testers. Also, since variable features as well as common ones are generally present in feature diagrams; this redundancy introduces unnecessary overhead in variability modeling. Furthermore, it is

indicated in [2] that feature models lack the necessary qualities such as scalability, traceability and consistency in variability modeling.

All these results suggest that variability needs to be considered at a wider scope than features' coverage. In order to achieve this, OVM has been proposed in [1] and *decisions* in [2] and [3]. Since the concept of *concerns* introduced in the scope of this work is related with *decisions*, they will be briefly discussed before delving into *concerns*.

Decisions are a broader version of variation points. *Decisions*, although expressing variability, consider more than one variation point at a time. They achieve this by gathering variability information at different abstraction layers under a single *decision* as explained in [2].

In Figure 2, a *decision* called “algorithm core” which represents variability across the real time scheduling algorithms (RTSA) domain can be observed.

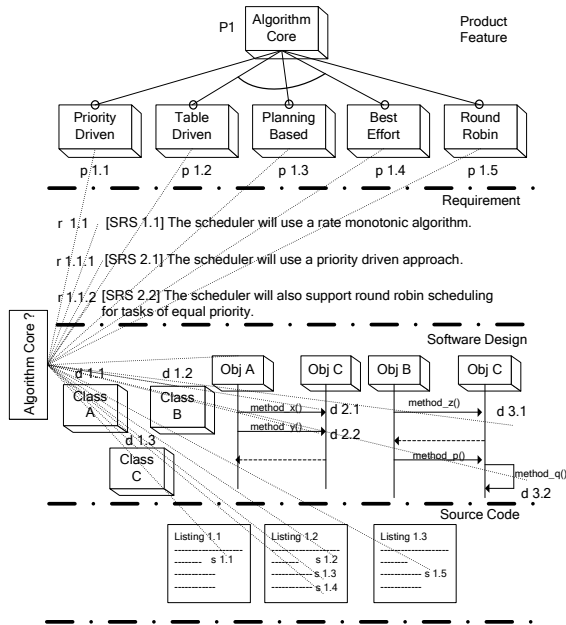


Figure 2 - “Algorithm core” decision

As could be deduced from the name, a *decision* needs to resolve the variability information it represents and obtain a final value during application engineering. Generally a simple yes-no question represents a *decision*, however in some complicated cases such as in the presence of dependencies, expressions with some constraints are preferred. The “algorithm core” *decision* is linked to the artifacts that must be configured if the decision is made.

5. Variability modeling with concerns

5.1 Concerns

Decisions mainly address the problem of traceability of variability artifacts. However several open issues remain. First of all *decisions* indicate the points to be configured when a particular decision is made without providing information about the nature of variability configuration at that particular point. The variability information at the configuration point may or may not be straightforward to handle. Secondly, an underlying formal method for managing spatial variations hasn't been proposed.

In order to resolve the first open issue, *concerns* recognize that the variability information in a PL is made up of two complementary classes of horizontal and vertical variability. The former represents the variability that is present within a particular asset abstraction layer and the latter is related with the variability that is present between the asset abstraction layers. Unlike *decisions*, variability in *concerns* is considered in both of these classes. Furthermore, both horizontal and vertical variability are modeled using formal means as described in the section below.

A *concern* is defined as the ensemble of variability information that captures and represents variability using a context free grammar (CFG) in a PL. A *concern model* represents PL variability via the usage of *concerns*. *Concerns* represent four levels of vertical variability in this particular work. These are product features, requirements, design and source code. Extension of these layers for finer tuning and/or restructuring these layers to distinguish between PL variability and software variability (a quality lacking in conventional feature diagrams as indicated in [8]) is up to the modeler. In this work no such extension is provided, instead the approach to variability modeling using *concerns* is explained.

The representation of the variability information (which is captured by a *decision* in Figure 2) via the usage of a *concern* can be seen in Figure 3. All variability information at all *concern* layers is depicted by its symbolic name within each *concern*. Note that actual asset counterparts of these symbolic names are discernable in Figure 2. Variation points and variants are depicted using uppercase and lowercase letters respectively. Symbolic naming uses letters F, R, D, and C corresponding to features, requirements, design and code.

A new variation point is created by a number postfix (D1, D2...). Variants are created using a numbering scheme that introduces a level under the respective

variation point (e.g. variants under the variation point D1 are d1.1, d1.2...). It is discouraged to have a number prefix in capital letter symbolic names for reasons that will become clear in the section below. Any other naming scheme is acceptable as long as variation points and variants are distinguishable in the *concern* model and uniquely represent their actual real asset counterparts.

The diagram notation used in expressing variability relations is the same notation used in feature relations in section 2. It should be noted that *excludes* and *requires* relationships that extrude out of variants imply connections to several other variants in other concerns that cannot be depicted due space restrictions.

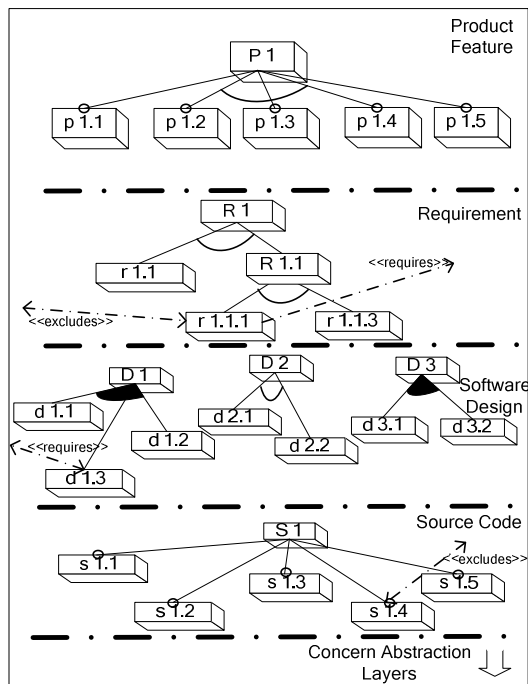


Figure 3 - "Algorithm core" concern

As noted in [12], several modeling approaches require more complex relationships such as "a *requires* b or c or d" among variants and that simple *requires/excludes* relationships are insufficient for expressing such complex ones. Modeling with *concerns* handles both simple and complex *requires/excludes* relationships since these are all handled on their own as explained in section 5.3.

5.2 Concerns and context free languages

In the context of OVM [1], it has been stated that the traceability between PL assets and variability information must be established via tools. Certainly,

such tool support can be made possible with a formal foundation for variability modeling and is essential for scalability purposes.

Evidently, traceability support is also valuable for scalability purposes, since if modeled properly, variability information for a PL of any size could be made readily available. Furthermore, simplicity of such a modeling approach is also essential for scalability as indicated in section 3. However simplicity should not allow for any compromise from the formality of a variability modeling approach, otherwise the correctness and integrity of the model would be hindered which in turn impairs scalability. In order to account for simplicity and disallow any impedance due to lack of formality, requirement or exclusion (either simple or complex) relationships are excluded from variability modeling at this stage.

A simple and convenient way to model variability in the absence of *requires/excludes* relationships is via the usage of a CFG. In [13], idea of utilizing a grammar has been employed in expressing feature models, however CFG's were not used and rather special notation and transformations have been defined to construct a grammar. In [12], an iterative grammar is used –with constructs such as one or more (+) and zero or more (*) – and a special notation is employed to denote optional features. Furthermore it has been indicated that CFG's are not suitable for feature diagrams with the complex relationships such as those mentioned in the section above. This discrepancy of CFG's and feature models is also applicable to the relation between *concerns* and CFG's since both *concerns* and features are used in expressing variability information. The modeling approach represented in this work recognizes this shortcoming and therefore excludes any kind of *requires/excludes* relationships from variability modeling at first.

Modeling variability via a CFG, justifies the motivation for the naming convention introduced in the section above. Variation points are non-terminals (upper case letters) and variants are terminals (lower case letters). Terminals form the alphabet of the context free language (CFL) defined by transformation rules of the CFG. A string of terminals is used in expressing a system that is a candidate for being a permissible system. For such a string to be valid, it should obey the rules of the CFG. The expression of the relations between features in Figure 1 via usage of a CFG is also applicable to relations between variation points and variants in a *concern*. Production rules (PR) of the CFG that represent each of the relations a) to e) in Figure 1 can be seen in (1) through (5).

$$\begin{aligned}
A &\rightarrow a \ b & (1) \\
A &\rightarrow a \ b \ A1 & (2) \\
A1 &\rightarrow c \mid \Lambda & (2) \\
A &\rightarrow a \mid b & (3) \\
A &\rightarrow a \ A1 \ A2 \mid \Lambda & (4) \\
A1 &\rightarrow d \mid \Lambda & (4) \\
A2 &\rightarrow b \mid c \mid \Lambda & (4) \\
A &\rightarrow a \mid b \mid a \ b & (5)
\end{aligned}$$

The corresponding relations to each of these PR's are explained in section 2 referring to Figure 1. In order to represent a particular relation by introducing an intermediate PR, it is possible to insert a new non-terminal by post-fixing a number to an existing non-terminal (such as A1 in (2)) or by extending the numbering level (such as A1.1). Moreover, to represent the choice of not including an optional variant into system description, a PR denoting the production of a null string as, $A \rightarrow \Lambda$, is used rather than a special notation as in [12] or [13].

Another subject to be considered is the representation technique used for expressing the transitions among layers in a *concern* using a CFG. In Figure 4, one can observe a partial *concern* with design and code layers.

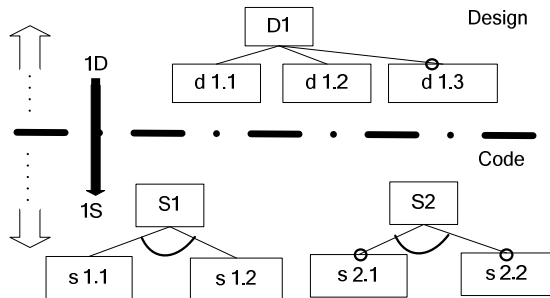


Figure 4 - Transition between concern layers

Since system description using a *concern* will cover specification variants from different abstraction layers, the CFL will contain strings that are made up of these variants. Thus the CFG corresponding to the CFL, needs to define PR's that lead to such composite variant strings. In this example a general case is assumed which relates D1 to both S1 and/or S2 or none of them. The notation that was forbidden in the section above with a number prefix in front of a nonterminal is used to describe *concern* layer transitions. The downward arrow in Figure 4 from 1D to 1S demonstrates this layer transition for which the governing PR's can be seen in (6).

$$\begin{aligned}
D1 &\rightarrow d1.1 \ d1.2 \ D \ 1.1 \ 1D \\
D \ 1.1 &\rightarrow d1.3 \mid \Lambda \\
1D &\rightarrow 1S \mid \Lambda \\
1S &\rightarrow S1 \mid S2 \mid S1 \ S2 \mid \Lambda \\
S1 &\rightarrow s1.1 \mid s1.2 \\
S2 &\rightarrow s2.1 \mid s2.2 \mid \Lambda
\end{aligned} \tag{6}$$

It must be noted that the PR for the null string production as, $1D \rightarrow \Lambda$, within the rule that realizes *concern* layer transition, namely $1D \rightarrow 1S \mid \Lambda$, is present to allow the freedom of not resolving variability at all artifact abstraction layers during application engineering. In this way, selective application of modeling with *concerns* is possible resulting in a modeling technique that is scalable within itself. If it is desired to model variability at several chosen layers –perhaps because due the special requirements or due partial availability of the variability model at a particular level– it is sufficient to choose the null string production at the appropriate layer transaction instance.

Multiple occurrences of a single variant throughout the *concern* model should be expected. As an example consider the hypothetical partial *concern* model in Figure 5.

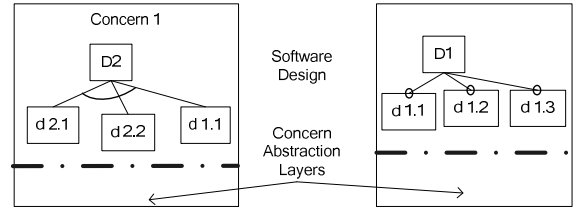


Figure 5 - Multiple occurrence of a variant

The grammar defining the partial *concern* model in Figure 5 can be seen in (7).

$$\begin{aligned}
D1 &\rightarrow d1.1 \ D1.1 \mid \Lambda \\
D1.1 &\rightarrow d1.2 \ D1.2 \mid \Lambda \\
D1.2 &\rightarrow d1.3 \mid \Lambda \\
D2 &\rightarrow d2.1 \mid d2.2 \mid d1.1
\end{aligned} \tag{7}$$

This situation may cause the following dilemma: Suppose the person modeling the system decides by barely considering D1, that d1.1 should be included in the system. However, if the very same person mistakenly thinks that either d2.1 or d2.2 should be included in the system, then since d1.1 is alternative to these; such a selection would not be possible. The modeler should take care when considering variation points with multiple occurrences or, the parser representing the grammar must and will warrant such an illegal selection.

Regarding the absence of *requires/excludes* relationships; it should first be stated that it is painful to represent these relationships using a CFG. To clarify this, let us consider *excludes* relationship shown in between variants d1.1 and s1.2 in Figure 6.

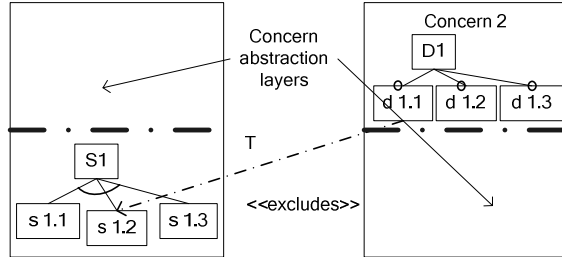


Figure 6 – Concern interrelationships

If this *excludes* relationship is desired to be shown by a CFG, the PR's which define the productions that are in the complement set of the production defined by (8) have to be included in the grammar. The reason for this is that, a CFG cannot specify invalid productions; it only demonstrates the valid ones.

$$\begin{aligned} D1 &\rightarrow d1.1 D1.1 T \mid \Lambda \\ D1.1 &\rightarrow d1.2 D1.2 \mid \Lambda \\ D1.2 &\rightarrow d1.3 \mid \Lambda \\ T &\rightarrow s1.2 \end{aligned} \quad (8)$$

Although it is simpler to model the *requires* relationship, this causes the increase of non-terminal symbols such as “T” in Figure 6. *Requires* is considered in the same category as *excludes* for this reason and also because of its conceptual proximity to *excludes*.

Secondly *requires/excludes* relationships tend to be complex in some cases as indicated in [12], and therefore it is more convenient to consider all such possible relationships altogether separately.

5.3 Using concerns for system specification

Modeling the multidimensional variability of a PL using a CFG is a good start for scalability since it is both simple, yet it is applicable to the growing/shrinking nature of the variability information. That being said, the ultimate goal of using *concerns* is to be able to use the complete variability information spread throughout the model to specify systems in the PL.

However, the CFL derived from variability relationships as described in the section above is incomplete since *requires/excludes* relationships are not considered. In the application model that is

suggested, a simple post-processor program that runs over the sequence generated using the CFG is used to check for the validity of such relations. These coexistence or coabsence relationships may be complex or simple. The underlying checker program consists simply of a sequence of some conditional checks, which can be automatically generated according to the rules defined anytime.

The application model of *concerns* can be seen in Figure 7. Variants that describe a potential system in the PL are chosen by the aid of the tree structure of *concerns*. The chosen variants form an initial string which is a potential system in the scope of the PL. Then this string of variants is fed to a parser that is created using the CFG which represents the *concern* model. This parser was implemented using *javacc* [14].

If the variant string is able to pass the checks of the CFG, this string enters the post-processing checks that take care of *requires/excludes* relationships. The post processor has been designed as a rule checker that allows the definition of custom *requires/excludes* relationships. If the variant string is able to pass this second check, then it describes a system that belongs to the PL according to the variability model's (the concern model) rules.

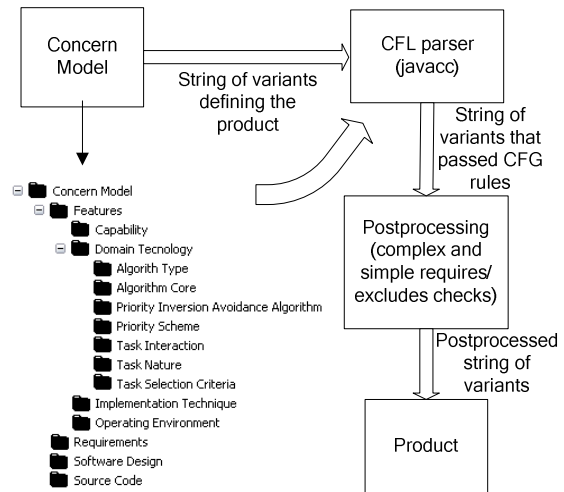


Figure 7 – Concerns usage in variability modeling

It should be noted that although the limitations of using a CFG in the modeling of PL variability have been acknowledged, it has been possible to overcome those limitations by separating variability modeling activities into two manageable phases as described above. This separation of activities is crucial for the overall modeling simplicity, which is part of the scalability definition mentioned in section 3.

6. Evaluation criteria for concerns

First expected benefit of the concern based approach is to obtain a model that describes fewer permitted systems compared to an approach that does not support such a modeling scheme. These systems are much closer to the systems that can be actually built. The reason for this is that *concerns* are able to reveal more variability-related relationships than features. The first proposed evaluation criteria is the permissibility ratio (PER) defined in (9) as the ratio of the number of systems that are acceptable as valid systems by the experts and users of the domain (N_a), to the total number of possible systems that can be generated using the particular modeling approach (N_t). Since it is the variability information that allows the creation of systems out of the PL, this criterion demonstrates the fitness of the variability modeling approach. Evidently, this ratio would ideally be 1.

$$PER = \frac{N_a}{N_t} \quad (9)$$

Although PER is helpful, it is painful to extract the correct N_a due to required workload of users and experts. A criterion that is easier to extract is the relative PER (RPER) defined in (10). RPER is the ratio of PER for two PL approaches (denoted by the subscripts in (10)). This ratio can be used to contrast two different variability modeling approaches. Note that since the same set of experts and users are expected to determine N_{a1} and N_{a2} , they will be equal, and therefore need not be calculated at all.

$$RPER_{1,2} = \frac{PER_1}{PER_2} = \frac{N_{t2}}{N_{t1}} \quad (10)$$

However, the calculation of N_t for both systems is not straightforward. To picture the situation it is helpful to consider two approaches with different variability modeling strategies. As part of this work, the contrasted PL approaches are bare FORM (model-1), and FORM with *concerns* (model-2). Since *concerns* take care of the variability modeling of a PL approach, generally they are intended to be used within an existing modeling approach by replacing the variability modeling technique of the original approach. However one cannot expect to extract any information from the N_t values' ratio for model-1 and model-2 that is calculated straightaway. Model-1 uses features to express variability and model-2 uses significantly more artifacts in addition to features to do so. This means

that N_{t2} can be either more or less than N_{t1} , since variability expression means of the approaches are different. In order to overcome this, the same means of variability modeling should be used (in this case the features). It can be asserted that the more $RPER_{1,2}$ is close to 0, the better the variability modeling approach of model 2.

Second benefit of the proposed approach is related with the first one and addresses feature interaction problem (FIP) in PL's. This problem arises when the coexistence of several features impedes the correct operation of a system. *Concerns* are expected to lower FIP due to multidimensional variability consideration. A variability model prohibits FIP via *excludes* relationships. So the more *excludes* there are, better the modeling approach in avoiding FIP.

The increase percentage of *excludes* relationships for a particular modeling artifact –as with the calculation of RPER– of two different variability modeling approaches can be used to contrast these approaches in terms of avoiding FIP (in this case it is the features). Thus, FIP avoidance percentage (A) is defined as in (11), where En_1 and En_2 are *excludes* relation count for the two modeling approaches.

$$A_{1,2} = \frac{En_2 - En_1}{En_1} \times 100 \quad (11)$$

7. Practical evaluation results

In order to assess the advantages of *concerns* in modeling variability in the light of the criteria introduced in the above section, an example domain of RTSA is modeled with bare FORM (model-1) and FORM with *concerns* for variability modeling (model-2). Source code variability modeling was not performed for model-2, due to resource constraints and advantage has been taken from the scalable applicability of *concerns*. In order to determine $RPER_{1,2}$ a simple tool developed in the context of this work has been used to automate the process of counting valid systems (N_{t1} and N_{t2}) according to system rules. Moreover, the number of *excludes* relations are readily available from the variability models which is necessary for determining $A_{1,2}$. The actual values of the evaluation criteria can be seen in (12) and (13).

$$RPER_{1,2} = 0.12 \quad (12)$$

$$A_{1,2} = 52.4 \% \quad (13)$$

The results show that even the partial application of *concerns* for variability modeling is helpful. The

permissibility ratio dropped by a factor of nearly nine, which means that model-2 represents the actual possible system combinations, nine times more accurately compared to model-1. The *excludes* number is nearly fifty percent higher compared to the original case, meaning interactions causing FIP diminished.

8. Conclusion

Complete and coherent modeling of variability is essential for a successful PL. However, such a task can easily grow out of control if not managed using a scalable modeling technique. Such a technique needs to be simple enough to enable straightforward modeling, yet powerful enough to formally specify variability elements. Furthermore it is desirable that such a modeling approach be scalable not only with respect to size requirements, but also with respect to temporal requirements of the PL effort.

Concerns are proposed in this work as a scalable means of modeling PL variability, which can be integrated to an existing PL approach by replacing its default variability modeling technique. *Concerns* recognize the multidimensional nature of variability spread to multiple artifacts that make up the PL and therefore are able to capture variability information at different abstraction layers. In our approach to modeling PL variability, in the first phase, *concerns* model basic variation point-variant relationships such as “or” groups and optional/alternative variants using a CFG and in the second phase, a post-processor covers the remaining simple or complex *excludes/requires* relationships. Separation of these two phases is essential for the simplicity and coherence of the approach and brings scalability with respect to model size, addressing the first and second requirements mentioned in section 3.

The third requirement in section 3 is satisfied by the method’s selective applicability to the different abstraction layers of variability information. A simple PR in the CFG allows for this selective application. This brings the temporal scalability to the approach since due to resource constraints; it may not be possible to derive the variability information in all artifacts of all asset abstraction layers.

As a means to assess the benefits of modeling with *concerns*, two novel evaluation criteria are introduced and the benefits of *concerns* are evaluated using these criteria on an example domain. Tools for supporting both the application model and measurement have been developed. *Concerns* proved to be helpful in variability modeling in the light of these criteria.

We propose that concerns can be an important foundation for creating domain specific languages (DSL). Future research will focus mainly on the usage of *concerns* for creating DSL’s. As part of future work, the concept of multidimensional variability modeling will be made available by a special tool that allows the custom specification of *concerns* (adjusting layer numbers and defining special interrelations). This tool will be used to express DSL’s based on concerns.

9. References

- [1] K. Pohl, G. Böckle, and F. van der Linden, *Software Product Line Engineering: Foundations, Principles, and Techniques*, Springer, 2005.
- [2] K. Berg, J. Bishop, and D. Muthig, “Tracing software product line variability: from problem to solution space,” *Proceedings of the 2005 annual research conference of the SAICSIT on IT research in developing countries*, White River, South Africa: SAICSIT, 2005, pp. 182-191.
- [3] D. Muthig and C. Atkinson, “Model-Driven Product Line Architectures,” *Software Product Lines*, 2002, pp. 79-90.
- [4] K. Kang, S. Cohen, J. Hess, W. Nowak, and S. Peterson, *Feature-Oriented Domain Analysis (FODA) Feasibility Study*, 1990.
- [5] K. Kang, S. Kim, J. Lee, K. Kim, E. Shin, and M. Huh, “FORM: A feature-oriented reuse method with domain-specific reference architectures,” *Annals of Software Engineering*, vol. 5, Jan. 1998, pp. 143-168.
- [6] H. Goma, *Designing Software Product Lines with UML: From Use Cases to Pattern-Based Software Architectures*, Addison Wesley Longman Publishing Co., Inc., 2004.
- [7] Available: <http://www.merriam-webster.com/dictionary>
- [8] A. Metzger and P. Heymans, *Comparing Feature Diagram Examples Found in the Research Literature*, Software Systems Engineering, 2007.
- [9] K. Czarnecki, S. Helsen, and U. Eisenecker, “Staged Configuration Using Feature Models,” *Software Product Lines*, 2004, pp. 266-283.
- [10] C. Gacek and M. Anastasopoulos, “Implementing product line variabilities,” *SIGSOFT Softw. Eng. Notes*, vol. 26, 2001, pp. 109-117.
- [11] M. Martinlasi, “Comparison of Software Product Line Architecture Design Methods: COPA, FAST, FORM, Kobra and QADA,” *Proceedings of the 26th International Conference on Software Engineering*, IEEE Computer Society, 2004, pp. 127-136.
- [12] D. Batory, “Feature Models, Grammars, and Propositional Formulas,” *Software Product Lines*, 2005, pp. 7-20.
- [13] M. de Jonge, J. Visser, and S. Implellang, “Grammars as Feature Diagrams,” *PROCEEDINGS OF WORKSHOP ON GENERATIVE PROGRAMMING*, vol. 12, 2002, pp. 23-24.
- [14] Available: <https://javacc.dev.java.net/>